

EarthScape Climate Agency

Big Data Climate Monitoring Platform

Project Overview & Technical Report

Platform	Flask (Python) web application
Database	MongoDB Atlas (cloud)
Purpose	Climate/environmental data ingestion, MapReduce-style processing, ML forecasting, and public weather monitoring
Roles	Administrator, Analyst
Frontend	Server-rendered Jinja2 + vanilla JS, Chart.js, Vanta.js/Three.js

1. Introduction

EarthScape Climate Agency is a Flask-based web platform for ingesting, processing, analyzing, and forecasting environmental data (weather station readings, satellite surface temperatures, and environmental sensor data such as CO2 levels). It combines a simulated Big Data pipeline (HDFS-style partitioned storage and MapReduce-style aggregation) with a machine learning layer (trend forecasting and anomaly detection) and a public-facing live weather page.

The application is built for a single-machine deployment backed by a cloud database (MongoDB Atlas), so it can run from any PC without local database installation, while still demonstrating the partitioning and map/shuffle/reduce concepts that a real Hadoop cluster would provide.

2. Core Features

- **Data Ingestion** — CSV upload or simulated data generation for three source types: weather stations, satellite readings, and environmental sensors.
- **Simulated HDFS storage** — ingested files are written into a date-partitioned directory layout (`raw/<source_type>/<year>/<month>/<day>/`), mirroring Hive/HDFS partition pruning, with write-once block files.
- **MapReduce-style processing** — per-group statistics (mean, min, max, standard deviation) and z-score anomaly detection, computed using Python's *multiprocessing.Pool* to parallelize the map phase across CPU cores.
- **Machine learning** — 7-day trend forecasting and anomaly detection built on scikit-learn, with an admin approval workflow before forecasts go public.
- **Live weather visualization** — a real-time animated sky widget (3D volumetric clouds via Vanta.js/Three.js) reacting to live weather data from the Open-Meteo API, with day/night lighting, rain, and lightning effects.
- **Alerting** — configurable thresholds and triggered-alert tracking with acknowledgement.
- **Support ticketing** — users can submit issues/feedback; administrators resolve them.
- **Role-based access control** — Administrator and Analyst roles with distinct permissions (see Section 4).

3. Technology Stack

Layer	Technology
Backend framework	Flask 3.0, Flask-Login (authentication), Flask-Mail (email)
Database	MongoDB Atlas (cloud), accessed via Flask-PyMongo
Data processing	Simulated HDFS (local partitioned files) + simulated MapReduce (Python multiprocessing)
Machine learning	scikit-learn (trend prediction, anomaly detection)
Frontend	Server-rendered Jinja2 templates, vanilla JavaScript

Charts	Chart.js
3D weather visual	Vanta.js (CLOUDS effect) + Three.js
Icons	Lucide
Scheduling	APScheduler (simulated real-time data streams)

4. User Roles & Permissions

The platform supports two roles. Any number of accounts may hold either role; new self-registrations default to Analyst, and an existing Administrator can promote or demote accounts from the User Management page.

Capability	Analyst	Administrator
Ingest data, run processing jobs	Yes	Yes
Generate ML forecasts	Yes	Yes
View alerts, acknowledge them	Yes	Yes
Submit support tickets	Yes	Yes
Approve / reject forecasts	No	Yes
Manage users & roles	No	Yes
Configure alert thresholds	No	Yes
Start / stop real-time streams	No	Yes
Resolve support tickets	No	Yes

No account may be deleted by either role — all changes are status updates (approve/reject, resolve/acknowledge), preserving a full audit trail.

5. Approval Workflow

Forecasts follow a review pipeline before they reach the public site:

- An Analyst ingests data and generates a 7-day forecast → status is set to *pending*.
- An Administrator reviews the pending forecast and approves or rejects it.
- Approved forecasts appear on the public landing page; rejected or pending forecasts never do.

6. Application Structure

The backend is organized as one Flask blueprint per feature area, all registered in `app/__init__.py`:

Blueprint	Purpose
public	Landing page and live weather (no login required)
auth	Login, register, forgot/reset password (shared split-panel UI)
dashboard	Role-specific dashboard with live stats and charts
ingestion	CSV upload / simulated data generation
processing	MapReduce-style job execution
ml	Trend prediction and anomaly detection
viz	Interactive chart visualizations
alerts	Threshold configuration and triggered alerts
predictions	7-day forecast generation and approval workflow
realtime	Simulated live data stream control
support	Support ticket submission and resolution
users	User management (administrator only)
settings	Profile and password management (all users)

7. Simulated Big Data Pipeline

This project does not run a real Hadoop cluster. Instead, `app/hdfs_sim/` and `app/mapreduce/` honestly simulate the two properties of HDFS/MapReduce that matter for the application's logic:

- **Partitioning** — ingested CSVs are written to `raw/<source_type>/<year>/<month>/<day>/*.csv`, matching the layout Hive/HDFS partition pruning uses.
- **Write-once blocks** — each ingested file is named uniquely and never overwritten, mirroring HDFS append-only block files.

- **Map** → **Shuffle** → **Reduce** — per-group statistics and z-score anomalies are computed from real ingested data using `multiprocessing.Pool` to parallelize the map phase across CPU cores.

This was a deliberate choice: running an actual Hadoop/HDFS cluster requires a Linux environment (or WSL2), Java, and substantial setup that is impractical for a single-machine deployment. The simulation preserves the same partitioning and map/shuffle/reduce shape, so the application logic is unaffected if a real cluster is substituted later.

8. User Interface

The interface was redesigned around a single, consistent design-token palette:

Token	Hex	Role (dark theme)
Background	#161A30	Page background, primary button fill
Surface	#31304D	Cards, panels, secondary accent
Muted text	#B6BBC4	Icons, secondary text, borders, active states
Primary text	#F0ECE5	Headings, body text, button label

- **Sidebar navigation** for authenticated pages, collapsing to a mobile drawer below 900px width.
- **Split-panel authentication** (login/register share one page with an animated illustration panel that swaps sides on tab switch; forgot/reset password follow the same layout).
- **Full-screen live weather hero** on the public landing page and dashboard, with weather-condition-aware backgrounds (clear sky uses a static gradient; cloudy/rain/storm use the 3D Vanta.js cloud field) and adaptive brand-text contrast via CSS blend modes.
- **Custom “E” monogram logo** and a matching illustration, both designed to the palette above.

9. Deployment Notes

- Data lives entirely in MongoDB Atlas (cloud), so the same account/data is visible from any machine running the app.
- Local files under `app/hdfs_sim/raw/` and `processed/` are machine-specific (the simulated HDFS storage) and are not required for the app to start.
- A non-SRV (standard) MongoDB connection string is recommended when the network's DNS cannot resolve `mongodb+srv://` SRV/TXT records, to avoid startup hangs.
- Full setup instructions are in `SETUP.md`; a manual testing checklist covering every page and role is in `TESTING.md`.